

FINAL YEAR PROJECT

AI Voice Appointment System

Final Technical & Implementation Report

A multilingual voice assistant that books, reschedules and cancels doctor appointments for a single clinic, with a Doctor Dashboard and a SuperAdmin Dashboard

Student name	
Registration number	
Supervisor	
Department	
University	
Academic year	
System version	1.0.0 (the version the API declares)
Report date	29 September 2026
Code described	The repository as it stands on the report date (github.com/HamzaTahir554/AI-voice-appointment-system)
Project status	Working prototype. The conversation, the appointment backend, Firestore, both dashboards, speech-to-text, text-to-speech and a microphone voice mode are implemented and tested; telephony (Asterisk/SIP) is not implemented , and patient notifications are queued but not delivered.

Contents

1	Executive Summary	12	Security
2	Introduction, Objectives and Scope	13	Testing
3	Technology Stack	14	Performance
4	System Architecture	15	User Flows
5	Speech and Telephony	16	Project Structure, Installation and Libraries
6	Intent Detection – mBERT	17	Limitations and Future Work
7	Dialogue Manager and Confidence Handling	18	Conclusion
8	Appointment Backend		References
9	Firebase Firestore	A	Appendix A – HTTP API
10	Ollama LLM Judge and Response Generator	B	Appendix B – Configuration
11	Doctor and SuperAdmin Dashboards		

Implementation Status at a Glance

Component	Status	Where
Fine-tuned mBERT intent detection (31 intents)	Implemented	intent_detection/ , dialog_manager/intent_router.py
Dialogue Manager	Implemented	dialog_manager/
Appointment Backend (validation, booking, leave)	Implemented	appointment_backend/
Firebase Firestore persistence	Implemented composite indexes declared, not deployed	firebase/ , firestore.indexes.json
Ollama judge / response generator and response validator	Implemented	ollama_judge/
ElevenLabs speech-to-text (realtime)	Implemented measured with synthetic callers	speech/stt.py
ElevenLabs text-to-speech (streaming)	Implemented measured	speech/tts.py
Microphone voice mode	Implemented tested with a virtual microphone	voice_app.py
Call endpoint for a telephony bridge	Implemented	api/voice_ws.py (/voice/call)
Asterisk / SIP telephony	Not implemented	Future work
Doctor Dashboard	Implemented	Dashboard/ , api/dashboard.py
SuperAdmin Dashboard (single clinic)	Implemented	Dashboard/ , api/admin.py
Appointment statistics	Implemented	appointment_backend/statistics.py
Doctor leave with automatic cancellation	Implemented	cancellation_service.py
Patient notifications	Queued, not delivered	firebase/notification_service.py
Dashboard authentication and roles	Prototype	api/auth.py

1. Executive Summary

Booking a doctor's appointment in Pakistan usually still means a phone call to a receptionist: the line is busy at peak hours, nobody answers after hours, and a receptionist juggling calls, walk-ins and paper diaries makes mistakes - double bookings, wrong days, missed cancellations. The **AI Voice Appointment System** automates that phone conversation. A patient speaks as they naturally would - English, Urdu, Roman Urdu, or a mixture such as *"Mje docter ke pas jana h"* - and the system works out what they want, asks only for what is missing, checks the doctor's real diary, and books, moves or cancels the appointment.

How it works. Speech is transcribed by **ElevenLabs Scribe v2 Realtime**. A **fine-tuned multilingual BERT (mBERT)** classifies each utterance into one of 31 intents (test accuracy 0.898, macro-F1 0.836 on 354 held-out sentences). A rule-based **Dialogue Manager** keeps the conversation state, fills the doctor/date/time slots, asks for confirmation and absorbs corrections. An **Appointment Backend** applies the clinic's rules and writes to **Firestore** inside a transaction, so a slot cannot be double-booked. Only then does a local language model (**Ollama, llama3.2**) phrase the reply; a **response validator** rejects any wording whose facts differ from the database, and the approved sentence is spoken by **ElevenLabs streaming TTS**.

Management. Doctors use a web **Doctor Dashboard** for their day's appointments, patients, weekly hours, leave, statistics and password. The clinic administrator uses a **SuperAdmin Dashboard** to add, edit, deactivate and remove doctors, set their usernames and passwords, manage leave, see and act on every appointment, edit the one clinic's details, and read clinic-wide and per-doctor statistics. It is a **single-clinic** system.

Status. Everything above is implemented and tested: 595 automated tests (582 passed, 13 live-service tests skipped, 0 failed) and 110 browser checks of the dashboards (all passed). Speech was measured against the real ElevenLabs service with synthetic callers: a caller who stops speaking hears the agent start after a median of 2.5 s (3.0-3.3 s in the microphone mode, where the language model words every reply).

Not implemented: the telephone line itself (Asterisk/SIP) and the delivery of patient notifications. A real person speaking into the microphone has not yet been measured.

Main contribution. A complete, working appointment assistant for Pakistani Urdu and Roman Urdu speech in which the machine-learning components decide *what the caller means* and *how the reply is worded*, while every fact the caller hears and every change to the diary comes from deterministic, tested code and the database.

2. Introduction, Objectives and Scope

2.1 Background

Most clinics in Pakistan take appointments by telephone and keep them in a paper register or a spreadsheet. This causes familiar problems:

- **Phone dependency.** Patients can book only while the line is staffed and free.
- **Receptionist workload.** The same questions - fees, timings, location, "is the doctor in tomorrow?" - take up most calls.
- **Errors.** Double bookings, wrong dates, and appointments that stay in the register after the patient cancelled.
- **Schedule changes.** When a doctor is away, every affected patient has to be found and called by hand.
- **Waiting.** Patients wait on hold, and wait at the clinic when bookings overlap.

Callers mix languages freely. The same request arrives as "I want an appointment", "مجھے اپائنٹمنٹ چاہیے", "Mujhe appointment chahiye" or "Mje docter ke pas jana h", with non-standard Roman Urdu spelling that keyword systems and English-only models handle poorly.

2.2 Problem statement

Build a system that can hold an appointment conversation with a patient in English, Urdu, Roman Urdu or mixed speech; understand what they want even when it is phrased informally; collect the details it needs; apply the clinic's real schedule and rules without ever inventing availability or confirming something that did not happen; and give the clinic's doctors and administrator a way to manage the same data.

2.3 Proposed solution

A pipeline in which each stage has one job: speech-to-text produces the caller's words; mBERT classifies the intent; the Dialogue Manager decides the next step; the Appointment Backend performs any operation against Firestore; the LLM phrases the result; the validator checks the phrasing against the result; text-to-speech speaks it. Two web dashboards read and change the same Firestore data through the same backend rules.

2.4 Objectives

Objective	Outcome
Voice-based appointment booking, cancellation and rescheduling	Met through the microphone mode and the call endpoint; not over a phone line
Multilingual intent detection (English, Urdu, Roman Urdu, mixed)	Met fine-tuned mBERT, 31 intents
Natural multi-turn conversation with confirmation and corrections	Met Dialogue Manager
Automated, rule-checked appointment management with no double bookings	Met validation gates and a Firestore transaction
Doctor schedule and leave management	Met both dashboards

Central SuperAdmin control of a single clinic	Met	
Real-time shared database	Met	Firebase Firestore
Spoken replies generated from verified data	Met	Ollama + validator + ElevenLabs TTS
Telephone deployment (Asterisk/SIP)	Not met	future work

2.5 Scope

User	In scope (implemented)
Patient (voice or text)	Book, cancel and reschedule an appointment; check their appointment; ask a doctor's fee, qualification, specialisation or general information; ask a doctor's availability on a day; ask the clinic's address and timings; greetings, thanks, goodbye, "repeat that", help; emergency escalation.
Doctor (Doctor Dashboard)	Today's appointments and who is due now; all own appointments (book, move, cancel, complete); own patients; own statistics; weekly working hours; leave (blocking dates cancels that day's appointments); own profile (not clinic details or username); own password; alert preferences; notifications queued for their patients.
SuperAdmin (SuperAdmin Dashboard)	Add, edit, activate/deactivate, remove (archive) and restore doctors; set a doctor's username and password; own password; any doctor's working hours and leave, with a preview of affected patients; every appointment in the clinic (cancel, move, complete); the clinic's name, address, city and phone; clinic-wide and per-doctor statistics.
Not in scope	Several clinics; payments; medical records; SMS or call delivery of notifications (records are queued only); the telephone line (Asterisk/SIP).

3. Technology Stack

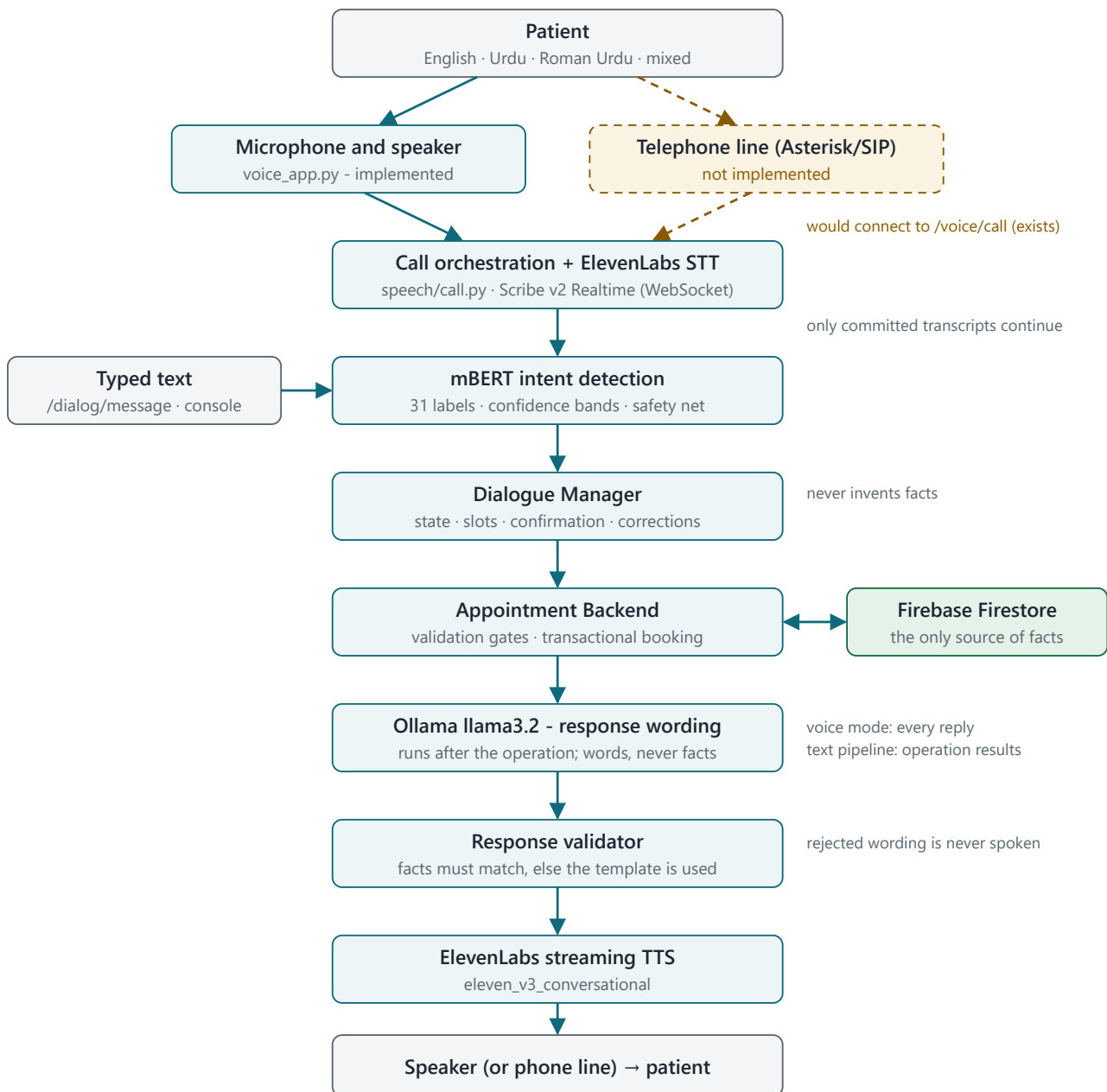
Every entry below is used by the current code; the versions are those installed on the development machine.

Category	Technology
Languages	Python 3.12 (backend, AI, speech); JavaScript, HTML and CSS (dashboards)
Intent detection	Fine-tuned mBERT (<code>bert-base-multilingual-cased</code>) with PyTorch 2.14 (CUDA 12.6) and Hugging Face Transformers 5.15
Speech-to-text	ElevenLabs Scribe v2 Realtime (WebSocket, <code>websockets</code> 15.0.1); Scribe v2 for whole recordings
Text-to-speech	ElevenLabs streaming TTS, model <code>eleven_v3_conversational</code> (over <code>httpx</code> 0.28.1)
Language model	Ollama 0.34.4 running <code>llama3.2</code> (3B) locally
Backend	FastAPI 0.141, Uvicorn 0.52, Pydantic 2.13, python-dotenv
Database	Google Cloud Firestore through the Firebase Admin SDK 7.5
Microphone and speaker	<code>sounddevice</code> 0.5.6 (PortAudio)
Dashboards	Plain HTML, CSS and JavaScript served by the API - no framework, no build step
Authentication	Own token sessions in the API; passwords hashed with PBKDF2-HMAC-SHA256
Data and evaluation	pandas, NumPy, scikit-learn, Matplotlib
Testing	Python <code>unittest</code> ; headless Google Chrome driven from Node.js for browser checks; pypdf
Version control	Git and GitHub
Telephony	Not implemented (Asterisk/SIP planned)

Hardware used for development, training and every measurement in this report: a desktop PC with an NVIDIA GeForce GTX 1660 SUPER (6 GB), on a network in Pakistan, with the free ElevenLabs plan.

4. System Architecture

4.1 The voice pipeline



One caller turn passes through every stage in order. The stages communicate by ordinary function calls inside one Python process, except for the three external services: ElevenLabs (STT over a WebSocket, TTS over HTTPS), Ollama (HTTP on the same machine) and Firestore (the Firebase Admin SDK).

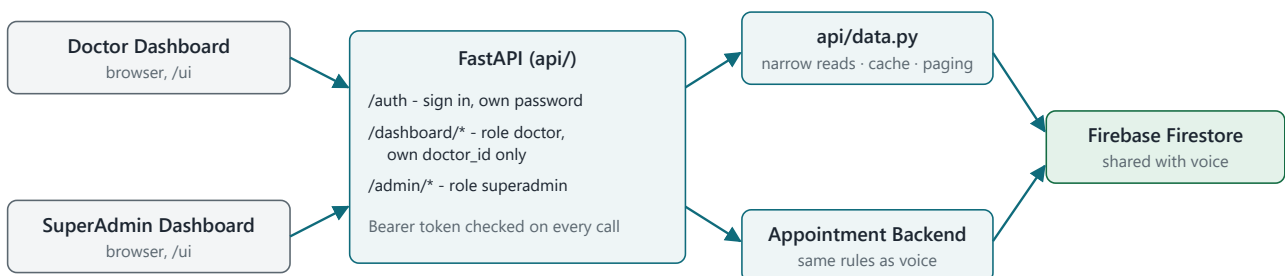
Component	Input → output	Code
Call orchestration	caller audio → committed transcripts; reply text → audio; decides <i>when</i> a turn happens, never <i>what</i> is said	speech/call.py
Speech-to-text	audio → partial and committed transcripts	speech/stt.py

Intent router	text → mBERT label, confidence, canonical intent, band	<code>dialog_manager/intent_router.py</code>
Dialogue Manager	intent + entities + session state → action, slots, reply template, backend result	<code>dialog_manager/dialog_manager.py</code>
Appointment Backend	booking request → <code>OperationResult</code> (success, data or error code)	<code>appointment_backend/</code>
Firestore services	one service per collection	<code>firebase/</code>
Voice pipeline	glues the stages; renders the reply in the caller's language; calls the judge	<code>voice_pipeline.py</code>
Judge and validator	result + template → approved sentence	<code>ollama_judge/</code>
Text-to-speech	approved sentence → streamed audio	<code>speech/tts.py</code>

4.2 Why the language model runs last

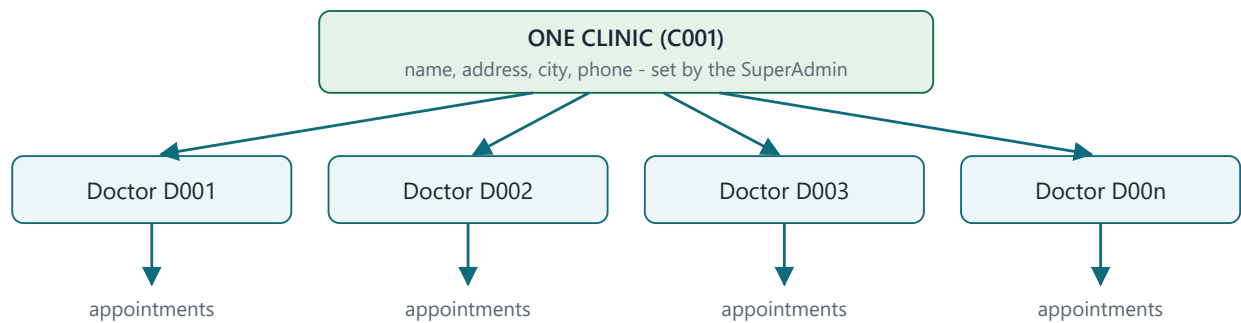
The language model is the only component that can make things up, so it is given no authority. By the time it runs, the appointment has already been booked or refused by deterministic code and the result is in Firestore. The model only chooses words, and the validator compares those words with the result: a time, date, doctor or appointment ID that is not in the result - or a claim of success after a failure - rejects the sentence, and the Dialogue Manager's own template is spoken instead. If Ollama is unavailable, the templates are used and the call continues.

4.3 Dashboard architecture



The browser never talks to Firebase and holds no credential or database address: it only calls the API with a session token. Writes that change appointments - booking, cancelling, moving, completing, leave - go through the same Appointment Backend as the voice assistant, so a doctor cannot bypass a rule the assistant has to follow. Because both paths use one Firestore database, a booking made by voice appears in the dashboard at its next 30-second refresh.

4.4 Single Clinic Architecture



The system serves **exactly one clinic**, configured by `CLINIC_ID` (default `C001`):

- The clinic's name, address, city and phone are **global**. The assistant reads them to callers, every dashboard screen shows them, and only the SuperAdmin can change them (`PATCH /admin/clinic`). A doctor who tries to set a clinic field on their own profile is refused with `403 clinic_readonly` .
- A doctor added by the SuperAdmin **belongs to the clinic automatically**; there is no clinic selector anywhere, and doctor details show the clinic's name without it being entered.
- There is **no way to add a second clinic**: the Clinic page shows one record, not a list (checked by the browser tests). Records left over from earlier multi-clinic test data can be merged into the one clinic with `POST /admin/clinic/consolidate` .
- Multi-clinic support is **not implemented** and not planned in this project.

5. Speech and Telephony

Speech is a layer around the text pipeline: it turns audio into the caller's words and the approved reply into audio, and changes nothing in between. The object that runs a call, `VoiceCall` (`speech/call.py`), is the same whether the audio comes from a microphone or, in future, a telephone bridge.

5.1 Speech-to-text (ElevenLabs)

Aspect	Implementation
Purpose	Turn the caller's speech into text for mBERT.
Model	<code>scribe_v2_realtime</code> over a WebSocket for calls; <code>scribe_v2</code> (batch) for whole recordings.
Input audio	<code>ulaw_8000</code> (telephone audio, sent as is) or <code>pcm_16000</code> (microphone), in 100 ms chunks.
Languages	<code>language_code=ur</code> with optional secondary languages. Urdu is returned in Urdu script and English words in Latin script, as spoken - nothing is translated. Roman Urdu speech is Urdu speech; it comes back mostly in Urdu script, which mBERT was trained on.
Turn-taking	A local speech gate holds back silence before the caller speaks (not paid for). ElevenLabs commits the sentence after 0.8 s of silence. Only committed transcripts go to mBERT ; partial transcripts are displayed and used to notice the caller interrupting.
Errors	Timeouts, rate limits, server errors and dropped connections are retried a bounded number of times; a dropped session is reopened (at most twice per call) and the sentence in progress is re-sent. ElevenLabs closes a session after ~15 s without audio (measured: 15.3 s), so 100 ms of silence is sent after 5 s idle. An empty transcript is never sent to mBERT; after two in a row the caller hears "Maazrat, mujhe clear sunai nahi diya. Dobara bol dein."
Security	The API key is read from <code>.env</code> and sent only in a request header; the <code>websockets</code> library's debug log, which prints headers, is held at INFO (a test caught it leaking the key).

Measured (29 September 2026, real ElevenLabs, synthetic caller voice, `docs/VOICE.md` section 9): **11 of 12** checks passed - ten sentences, of which nine were transcribed well enough for mBERT to find the right intent, plus two seconds of silence and of line noise, which correctly produced no transcript. The nine include "Mje docter ke pas jana h" ("جی، doctor کے پاس جانا ہے" → `book_appointment`); the failure was "fee kitni" heard as "فکت نہیں". The committed transcript arrived **1.5-1.8 s** after the caller stopped (0.8 s of that is the end-of-sentence pause); a session connects in about 0.8 s. The synthetic caller also mis-heard time words ("kal" → "Cole", "kal ke liye" → "Calcutta").

5.2 Text-to-speech (ElevenLabs)

Aspect	Implementation
Purpose	Speak the final, validated reply - exactly the pipeline's <code>response</code> , not rewritten (a test compares the synthesised text with each turn's reply).
Model	<code>eleven_v3_conversational</code> , chosen by measurement (below). Voice: the built-in "Sarah" voice; the free plan refuses Voice Library voices through the API (<code>402 paid_plan_required</code>).

Streaming	The <code>/stream</code> endpoint: audio is played as it arrives. <code>ulaw_8000</code> for a phone line, <code>pcm_22050</code> for the speaker.
Errors	Retried only before the first audio byte - never restarted mid-sentence. If TTS fails, the reply is still shown as text and the call continues; an appointment is never booked twice because speech failed (tested).
Cost	Only fixed phrases (greeting, farewell, apologies) are cached as audio.

Measured: first audio after a median of **818 ms** (`eleven_flash_v2_5`) and **824 ms**

(`eleven_v3_conversational`), 11 sentences × 2 runs. Each sample was transcribed back: Flash turned “4 baje” into “5 baje” and “available nahi” into “unavailable nahi” - a changed time and a reversed meaning - where v3 Conversational was correct, and Flash v2.5 does not list Urdu. Pronunciation of Roman Urdu by the English-trained “Sarah” voice has **not been judged by a listener**.

5.3 Microphone voice mode

`python voice_app.py` puts the computer's microphone and speaker in the telephone line's place - the development stand-in until Asterisk/SIP exists.

```
Microphone (16 kHz PCM) → ElevenLabs STT → mBERT → Dialogue Manager → Appointment Backend
→ Firestore → Ollama wording → validator → ElevenLabs streaming TTS (22 kHz PCM) → Speaker
```

- **Devices:** the default microphone and speaker, or any chosen by number or part of the name (`MICROPHONE_DEVICE` , `SPEAKER_DEVICE` , `--list-devices`). A missing, busy, muted or unplugged device is reported in words, not a traceback; an unplugged microphone is waited for 30 s. `--mic-test` shows the level against the speech threshold without sending anything.
- **End of speech:** voice activity (default: pauses under 0.8 s stay inside the sentence) or push-to-talk (`--ptt` : SPACE to start, SPACE to stop).
- **Feedback prevention:** through loudspeakers the microphone is muted while the agent can be heard, plus 300 ms of room echo, by feeding silence instead of the microphone's audio - so the agent never transcribes itself. The cost is that it cannot be interrupted; with headphones `VOICE_BARGE_IN=true` keeps listening and uses the call's barge-in (tested with simulated audio only).
- **Session:** one dialogue session for the whole conversation; it ends on goodbye or on q / Esc / Ctrl+C.
- **Every reply through Ollama:** in voice mode every reply - including slot questions - is worded by the model and checked by the validator (section 10).
- **Debug view** (`--debug`): transcript, intent and confidence, dialogue action, database result, whether the model's wording was approved or rejected and why, the spoken reply, and the time of every stage.

Measured live with real ElevenLabs, mBERT, llama3.2 and the real speaker, but a **virtual microphone** (a synthetic caller at real-time pace) and the in-memory clinic: in 3 six-turn conversations every reply was spoken; two conversations completed a booking by voice; one failed because “kal ke liye” was transcribed as “Calcutta”. The critical sentence “Mje docter ke pas jana h” was classified `book_appointment` in all three. Caller stops → agent audio: median **3.05 s** over 18 turns. **Not yet tested: a person speaking into the real microphone.** Both microphones on the development machine were opened and their levels measured, nothing more.

5.4 Asterisk / SIP telephony

Status: Not implemented - future work. There is no Asterisk configuration, SIP account or telephony bridge in the project, and no telephone call has been made to the system.

What exists is the interface a bridge would use: a WebSocket endpoint, `/voice/call` (`api/voice_ws.py`), that accepts 8 kHz μ -law audio (or 16-bit PCM), runs the full pipeline and sends back the agent's audio plus `stop_playback` and `hangup` events. It is protected by a shared token (`VOICE_GATEWAY_TOKEN`, or local connections only when unset) and is covered by automated tests with simulated audio. An Asterisk bridge (for example ARI External Media or AudioSocket) would forward the caller's audio to this socket and play what comes back; `docs/VOICE.md` section 5 describes what it must do.

6. Intent Detection – mBERT

6.1 Why mBERT

Callers mix English, Urdu script and Roman Urdu in one sentence and spell Roman Urdu freely (“mujhe”, “mje”, “mujhy”). Multilingual BERT (`bert-base-multilingual-cased`) was pre-trained on Wikipedia in 104 languages including Urdu and English, with one WordPiece vocabulary shared by all of them, so a single model can read all three forms. It is small enough to fine-tune on a 6 GB GPU and answers in about 8 ms per sentence on that GPU. The *cased* variant is used because the text is kept as the caller said it.

6.2 Dataset

Source (in Data/)	Rows	Notes
<code>dataset.csv</code>	405	English/Roman Urdu queries with a parallel Urdu-script column; 9 coarse labels, 384 of them “book”
<code>dataset2.csv</code>	2,159	19 columns; 2,000 duplicated (text, intent) pairs
<code>dataset3.csv</code>	1,500	English; 1,425 fully duplicated rows
<code>synthetic_intents.csv</code>	2,377	Generated by <code>generate_data.py</code> for all 31 intents: includes 127 hand-written natural phrasings and STT-style variants, 507 Roman Urdu spelling variants, 38 booking-shaped emergencies and 159 boundary phrasings added after an audit

After harvesting the parallel Urdu columns there were 8,346 rows; exact-duplicate removal left **3,461 unique sentences** in 2,991 leakage-proof groups. By script: 1,242 Roman Urdu, 1,231 English, 987 Urdu script (and 1 Devanagari). Most sentences (2,374 of 3,461) come from the generated set - an honest limitation discussed in section 17.

6.3 Preprocessing and tokenization

1. Detect the text and intent columns of each file automatically (the four files use different schemas).
2. Harvest parallel Urdu-script columns as extra samples.
3. Map every raw label onto one 31-intent taxonomy.
4. Normalise Unicode (NFKC) and whitespace only. No lower-casing, stop-word removal or stemming: mBERT is cased and a question mark is a real signal for question intents.
5. Remove exact duplicates.
6. Split 80/10/10, stratified by intent and **grouped**, so near-identical variants never straddle train and test: **2,760 train / 347 validation / 354 test**, with 0 texts shared between train and test. Sentences that also appear in the evaluation sets were held out of training.

Tokenization uses mBERT's own WordPiece tokenizer with a maximum length of 64 tokens (padded/truncated), which covers a spoken sentence.

6.4 Fine-tuning

Setting	Value	Setting	Value
Base model	<code>bert-base-multilingual-cased</code>	Epochs	20 (early stopping, patience 5)

Classifier	linear head, 31 outputs	Best epoch	19 (validation macro-F1 0.8659)
Batch	16, effective 32 (2-step accumulation)	Loss	cross-entropy with class weights (power 0.5)
Learning rate	2e-5, 10 % warm-up	Precision	mixed (fp16)
Weight decay	0.01	Gradient clipping	1.0
Hardware	GTX 1660 SUPER, 6 GB	Time	≈ 233 s per epoch

6.5 Results on the held-out test set

Evaluated on 354 test sentences never seen in training (`intent_detection/outputs/` , 14 September 2026 - the model in use).

Metric	Macro	Weighted
Accuracy		0.8983
Precision	0.8680	0.9097
Recall	0.8404	0.8983
F1-score	0.8362	0.8937

Strongest intents (F1 1.00): `change_date`, `change_doctor`, `greeting`, `doctor_unavailable` and the patient-detail intents; `book_appointment` 0.97 on 148 sentences. **Weakest:** `clinic_timing` 0.40, `goodbye` 0.50 (recall 0.33), `repeat_information` 0.57, `help` 0.62, `change_time` 0.67, `emergency` 0.71 (recall 0.55). Most common confusions: `change_time` → `reschedule` (2 of 6), `goodbye` → `repeat_information` (2 of 6), `cancel` → `book` (2 of 8), `emergency` → `book` (2 of 11). Several intents have fewer than ten test sentences, so their scores move a lot with one sentence. The weak spots are covered by rules in the Dialogue Manager: an emergency safety net, and corrections for the confusions seen in live calls (section 7).

Minimum confidence to accept	Sentences kept	Accuracy on kept
0.50	98.3 %	0.908
0.60 (used)	97.7 %	0.913
0.80 (used)	93.8 %	0.940
0.90	92.1 %	0.951

Mean confidence was 0.983 on correct predictions and 0.807 on wrong ones, which is why confidence alone cannot catch every error.

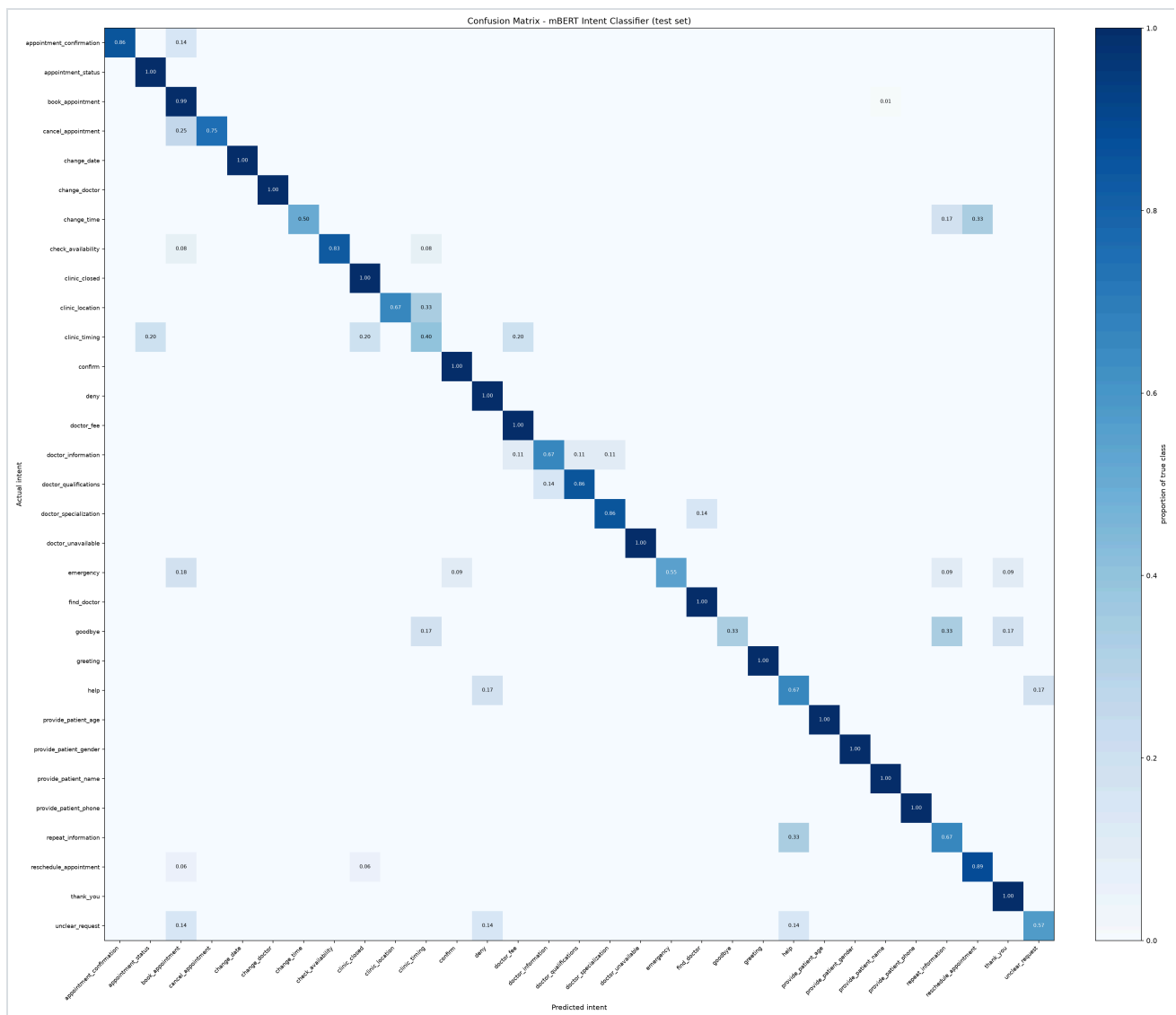


Figure 6.1 - Normalised confusion matrix of the model in use, test set (354 sentences). The image is high resolution: zoom in to read the intent labels.

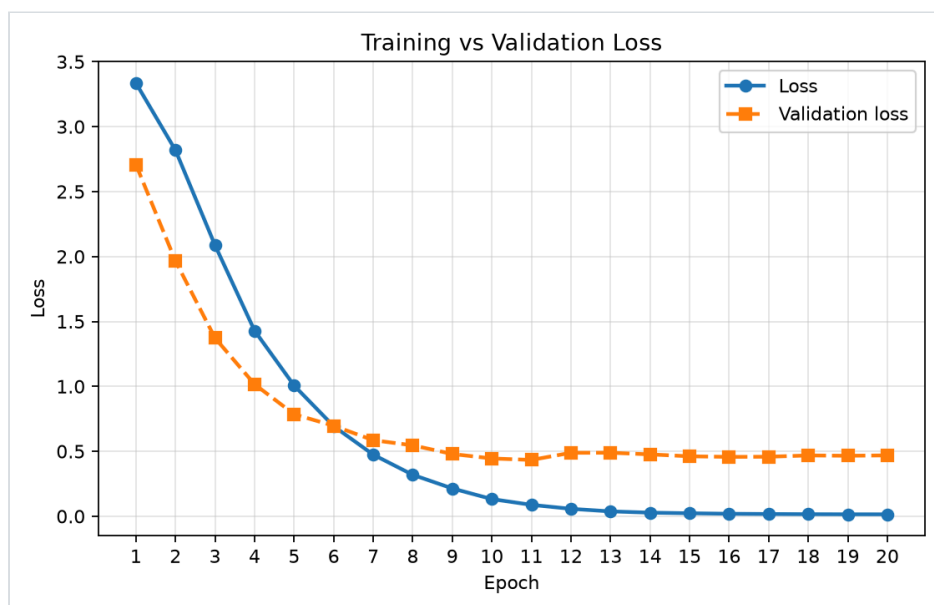


Figure 6.2 - Training and validation loss per epoch.

6.6 Audit and hold-out sets

Beyond the test split, the model was checked on hand-written evaluation sets (`reports/intent_audit/`). An audit found that 30 of the 164 audit sentences and 1 of the 41 hold-out sentences had also been in the training data; they were removed and the model retrained - the current model. Its results with no leakage:

Set	Sentences	Accuracy	By language
Audit	164	0.927	English 0.944, Roman Urdu 0.936, Urdu 0.903, mixed 2 of 3
Hold-out	41	0.878	English 1.00, Roman Urdu 0.913, Urdu 4 of 6, mixed 1 of 2
Challenge (look-alikes)	29	1.000	Roman Urdu

The previous model scored 0.916 accuracy on its own, different 333-sentence test split; that figure is not comparable with the current 0.898.

6.7 The 31 intents

mBERT label	Handled as	Example from the training data
book_appointment	book	mujhy dctr se check karwana tha
cancel_appointment	cancel	meri apointment khatam kar dein
reschedule_appointment	reschedule	Move my appointment to Friday at 12 PM
change_date	reschedule (or edit a booking draft)	Mujhe doosri date chahiye
change_time	reschedule (or edit a booking draft)	mujhe doosra time de dein
change_doctor	reschedule (or edit a booking draft)	Please switch me to another doctor
appointment_status	check appointment	Kya mera appointment lag gaya hai
appointment_confirmation	confirm	Wo date theek hai, confirm karein
check_availability	doctor availability	Doctor kab available hote hain
doctor_unavailable	doctor availability	Is Dr Fatima out of station
find_doctor	doctor information (list)	Mujhe dermatologist chahiye
doctor_information	doctor information	Dr Ayesha kaun hain
doctor_fee	doctor information (fee)	Dr Usman ki fees kya hai
doctor_qualifications	doctor information (qualification)	Dr Fatima ki degree kya hai
doctor_specialization	doctor information (specialisation)	What kind of doctor is Dr Sara
clinic_location	clinic information (address)	Kis ilaqe mein hai clinic
clinic_timing	clinic information (timing)	What are your working hours
clinic_closed	clinic information (closed days)	Kya clinic aaj band hai
emergency	emergency - escalate	Mere abbu gir gaye hain, abhi madad chahiye
greeting	greeting	Aap kaise hain

goodbye	goodbye - end the conversation	Theek hai, khuda hafiz
thank_you	thanks	Waqf dene ka shukriya
help	help	Kuch samajh nahi aa raha
confirm	yes	theek hai kar do
deny	no	No that is wrong
repeat_information	repeat the last reply	Maaf kijiye, sun nahi paya
unclear_request	unknown - clarify	Chhorein koi baat nahi
provide_patient_name	caller gives information	ji mera naam Usman hai
provide_patient_phone	caller gives information	My mobile is 0300 1234567
provide_patient_age	caller gives information	Meri umar 60 saal hai
provide_patient_gender	caller gives information	Yeh khatoon ke liye hai

The 31 labels map onto 17 canonical intents the Dialogue Manager acts on (`INTENT_MAPPING` in `config.py`), so the model can be retrained with different labels without touching the dialogue logic.

7. Dialogue Manager and Confidence Handling

mBERT answers “what does the caller want?”; the Dialogue Manager (`dialog_manager/`) answers “what should happen next?”. It is deterministic and never invents a fact: availability, fees, addresses and appointment IDs all come from the database.

7.1 Conversation state

Each session (one call or chat) keeps: the patient, the current intent and named dialogue state (idle, booking, confirmation, cancel, reschedule, status, doctor information, clarification), the slots collected so far, the slot being asked for, a **pending intent** (a question parked until a missing detail arrives), whether a yes/no confirmation is awaited, the reply language, and the last 40 turns. Sessions expire after 15 minutes of inactivity. Every message, with its intent and confidence, is logged to Firestore (`conversation_sessions` , `conversation_messages`).

7.2 One turn

1. **Perception:** mBERT intent and confidence; rule-based entities; yes/no.
2. **A pending confirmation outranks everything:** the caller is answering “shall I book it?”.
3. **Effective intent:** chosen from the prediction *and* the context (7.4).
4. **Medium-confidence check:** a guess that would start a new task with nothing to back it up is confirmed first (7.5).
5. **Intent switch:** a confident new task abandons the old draft (bar 0.75).
6. **Slots:** store what the caller said; a side question about another doctor does not change the doctor being booked.
7. **Dispatch** to the handler, which asks for the next missing slot, asks for confirmation, or calls the backend.

7.3 Slot filling and entities

Task	Required slots (asked in this order)
Book	doctor, date, time (patient name and phone optional)
Cancel	appointment ID - or the caller's single upcoming appointment when the caller is known
Reschedule	appointment ID (same rule), new date, new time
Check appointment	none - found from the caller
Doctor availability	doctor, date

Entities are extracted by deterministic rules (`validators.py`), because “kal 4 baje” must always mean the same date and time: doctor names and their aliases in English and Urdu script; *aaj*, *kal*, *parson*, weekdays in English, Roman Urdu and Urdu, and calendar dates; times such as “4 baje”, “shaam 5”, “5 PM” and Urdu digits and number words, with a bare 1-8 o'clock read as afternoon (clinic hours); appointment IDs; phone numbers; names. All dates are resolved in Asia/Karachi time.

7.4 Context-aware decisions

- **Short answers continue the task.** When the system asked “which day?” and the caller says “kal”, the model may predict anything; the date entity and the open booking decide.
- **Corrections:** “nahi Dr Asim” replaces the doctor; the reply acknowledges it (“Theek hai ji”).
- **Draft edits:** inside a booking, `change_date` / `change_time` / `change_doctor` edit the draft instead of starting a reschedule (which would ask for an appointment ID the caller does not have - seen in a live test).
- **Side questions:** “Dr Ahmed ki fee kitni hai?” during a booking is answered, and the booking question is asked again in the same reply.
- **Moving one's own appointment:** “Meri appointment Friday ko 5 baje kar dein” reads as a booking to mBERT; for a caller who has an upcoming appointment it is treated as a reschedule.
- **Confirmation:** nothing is booked, moved or cancelled without an explicit yes; “ہاں، کر دیں” counts as yes even when mBERT reads it as `change_date`.
- **Clarification:** after two failed attempts the assistant stops asking open questions and reads a short menu of what it can do.

Real turns from the current system (text input, in-memory clinic) - the model's label on its own is often wrong for a one-word answer, and the context carries the conversation:

System asked	Caller said	mBERT alone	What happened
Which day?	kal	<code>change_date</code> 0.34	date = tomorrow → asks the time
What time?	5 baje	<code>unclear_request</code> 0.89	time = 17:00 → asks to confirm
Appointment ID?	APT123	<code>provide_patient_phone</code> 0.97	ID taken → continues the reschedule
Which day?	Friday	<code>greeting</code> 0.95	date = Friday → asks the time
Which day? (booking Dr Ahmed)	nahi Dr Asim	<code>doctor_unavailable</code> 0.67	doctor corrected to Dr Asim → asks the day

7.5 Confidence handling

Band	Policy
High (≥ 0.80)	Act on the intent.
Medium (0.60-0.80)	Believe it only if the conversation supports it: a task is already open, or the sentence contains a doctor, date, time or appointment ID. Otherwise ask one short question - “Ji, aap appointment cancel karna chahte hain?” - instead of acting on a 0.65 guess.
Low (< 0.60)	Never act; the intent is unknown and the caller is asked what they meant - unless the corroboration rule below applies.
Switching task	Abandoning a task in progress needs ≥ 0.75 .

The thresholds are configurable (`HIGH_CONFIDENCE` , `CONFIDENCE_THRESHOLD` , `INTENT_SWITCH_THRESHOLD`).

Three rules run on the router's output:

- **Emergency safety net.** Acute danger phrases (“saans nahi aa rahi”, “khoon beh raha hai”, “behosh”) escalate to emergency whatever the model said - added after the model read “Bohot khoon beh raha

hai" as `thank_you` 0.97. The caller is told to call emergency services or go to the nearest hospital.

- **Booking corroboration.** A low or medium guess is overridden to `book_appointment` only when the sentence has a subject (doctor, appointment), an action (jana, milna, chahiye...) and no competing vocabulary (fee, cancel, change, emergency...). It was written for "Mje docter ke pas jana h", which an earlier model classified as `change_doctor` 0.41. The current model classifies it as `book_appointment` with confidence 1.00; the rule remains as a safety net.
- **Known confusions** found in live calls: a farewell read as thanks, and availability asked in Urdu script.

8. Appointment Backend

The only code that creates or changes appointments (`appointment_backend/`). The voice assistant, the Doctor Dashboard and the SuperAdmin Dashboard all go through it. Every operation returns an `OperationResult` : success, the data the reply needs, or a precise error code.

8.1 Operations

Operation	What it does
Book	Runs the gates below and writes a <code>confirmed</code> appointment with an ID such as <code>APT48D0EE</code> .
Cancel	Changes the status to <code>cancelled</code> (records are never deleted); checks the appointment exists, belongs to the caller and is still live.
Reschedule	Same checks, then the new date and time through the date, time and slot gates; the record is updated in place with status <code>rescheduled</code> . (Unlike a new booking, this write is not inside a transaction - see section 17.)
Check	A caller's upcoming appointments, or "could this be booked?" without writing.
Availability	Free slots for a doctor and date: the doctor's schedule for that weekday (20-minute slots in the demo schedule, 16:00-20:00, Monday-Saturday) minus leave and live appointments.
Alternatives	When a slot is taken, the three free slots <i>nearest the requested time</i> - "5:00 PM is already taken. Dr Asim Raza has 4:20 PM, 4:40 PM or 5:20 PM free."
Complete	Marks a consultation finished (dashboards).
Doctor leave	Blocks a date and cancels every live appointment on it (8.4).

8.2 Validation gates (booking), in order

1. The patient exists.
2. The doctor exists and is active (a deactivated doctor cannot be booked).
3. The clinic exists.
4. The date is a real date and not in the past (Asia/Karachi).
5. The time is a valid `HH:MM` .
6. No duplicate: the same patient with the same doctor on the same day is refused.
7. The doctor works that day, is not on leave, and the time is inside the working hours.
8. The slot is free.
9. The write re-checks the slot **inside a Firestore transaction** (`reserve_slot`): if someone took it between the check and the write, the booking fails with alternatives instead of creating a double booking.

Error codes: `patient_not_found` , `doctor_not_found` , `clinic_not_found` , `invalid_date` , `date_in_past` , `invalid_time` , `duplicate_appointment` , `doctor_not_working` , `doctor_unavailable` , `outside_working_hours` , `slot_unavailable` , `appointment_not_found` , `not_your_appointment` , `already_cancelled` , `already_completed` , `backend_unavailable` .

8.3 Appointment statuses

Status	Meaning
pending	Defined for unconfirmed bookings; the current flows confirm before writing, so new bookings are confirmed
confirmed	Booked
rescheduled	Live, moved to a new time
completed	Consultation finished
cancelled	Cancelled by the patient or the clinic
cancelled_by_doctor	Cancelled because the doctor blocked the day

Pending, confirmed and rescheduled appointments hold their slot; the others free it.

8.4 Doctor leave and patient notification

Doctor (own dashboard) or SuperAdmin blocks a date, with a reason
↓ the SuperAdmin first sees a preview: who will be affected
Date recorded in doctor_unavailability - the doctor can no longer be booked that day
↓
Every live appointment of that doctor on that date → status cancelled_by_doctor
↓ (records kept, with the reason and the time)
One notification per patient written to notifications:
"Dr Hamza Ali is not available on 2026-10-01, so your appointment at 17:00
has been cancelled (Medical conference). Please call again to book another date."
↓
Shown on the doctor's Notifications page

Delivery is not implemented. Notifications are queued in Firestore; nothing is sent by SMS or phone call. A `NotificationSender` interface exists, and `NOTIFICATION_PROVIDER` names a provider, but no provider is implemented, so the API reports the notifications as "queued only".

9. Firebase Firestore

Firestore is the single source of truth. It was chosen because it needs no server to run, is reachable from Python through the official Admin SDK, supports transactions (used to prevent double bookings) and lets the voice assistant and the dashboards share one live database. Without credentials the system falls back to an in-memory database with the same interface, used by the automated tests so they never touch the live project.

9.1 Collections

Collection	Contents
<code>clinics</code>	The one clinic: name, address, city, phone
<code>doctors</code>	Name, specialisation, qualification, fee, experience, phone, aliases (for recognising the name in speech), active/archived flags, clinic
<code>schedules</code>	Working hours per doctor and weekday: sessions, slot length
<code>doctor_unavailability</code>	Blocked dates (leave) with reason
<code>patients</code>	Patient ID, name, phone
<code>appointments</code>	Appointment ID, patient, doctor, clinic, date, time, status, cancellation details
<code>notifications</code>	Messages queued for patients (doctor leave, clinic cancellations), read flags
<code>conversation_sessions</code>	One record per conversation
<code>conversation_messages</code>	Every turn, with intent and confidence
<code>dashboard_accounts</code>	Sign-in accounts: username, role, PBKDF2 password hash

Relationships are by ID: an appointment carries `patient_id`, `doctor_id` and `clinic_id`; a schedule and a leave record carry `doctor_id`. The demo data (`firebase/seed_data.py`) is one clinic, four doctors, their Monday-Saturday schedules and one patient; it is written only to the in-memory database or when run explicitly.

9.2 Read performance

The dashboards were measured to be slow on the live project (sign-in to a usable doctor dashboard: 7.3 s) and reworked (`api/data.py` , `docs/PERFORMANCE.md`):

- **Query filtering:** screens read only their records (by doctor and date) instead of whole collections.
- **Batching:** the patients on a page are fetched by ID in one call.
- **Counting without downloading:** totals and statistics use Firestore `count()` aggregations.
- **Parallel reads** inside a request; **server-side paging** by date.
- **Caching:** the clinic record and the doctor register are cached in the API for one minute and dropped at once by any write through the API; appointments, patients and notifications are never cached, and the booking rules never read the cache.
- **Indexes:** four composite indexes are declared in `firestore.indexes.json`. They have **not been deployed** to the live project; until they are, each query that needs one falls back to a simpler query plus filtering, with the same result.

Measured on the live project (median of three runs): doctor sign-in to a usable dashboard 7.3 s → 1.2 s; the 30-second refresh 3.4 s → 0.57 s (4 requests → 1); administrator overview 5.7 s → 1.7 s; every page once 40.8 s → 12.0 s. On a simulated clinic of 20,000 appointments the same pages read 9,240 documents instead of 157,429. From this machine one Firestore round trip takes about 0.55 s, and for long stretches about 2.35 s - network latency outside the application.

10. Ollama LLM Judge and Response Generator

10.1 Role

A local model, **llama3.2** (3B) served by Ollama, words the reply. It is **not** the source of truth: it cannot read or write the database, and it runs after the Dialogue Manager and the backend have finished. Settings: temperature 0.2, at most 200 generated tokens, JSON output, 30 s timeout, address

`http://127.0.0.1:11434` .

10.2 Two uses

Use	Input	Check
Judge - turns with a database result (book, cancel, reschedule, alternatives, availability, errors)	The caller's words, intent, confidence, slots and the backend result as JSON, the reply language, and worked examples. The model returns a sentence and a decision.	<code>validate_llm_response</code> : against the backend result
Rewording - every other reply, in voice mode (<code>VOICE_OLLAMA_EVERY_TURN</code>)	The Dialogue Manager's own sentence and the reply language, with examples in that language only.	<code>validate_rewording</code> : against that sentence

The typed-text pipeline uses the judge only; slot questions there come from templates, because a 3B model's Roman Urdu cannot be checked without a reference.

10.3 Hallucination prevention - the validator

The prompt asks the model not to invent facts; the validator enforces it. A generated sentence is **rejected** when:

- it mentions a time, date or appointment ID that is not in the backend result ("4 bajē" is accepted for 16:00; "17 bajē" is rejected as unnatural Urdu);
- it says *kal*, a weekday or a day of the month that is not the backend's date;
- it claims success ("booked", "ho gayi", "cancel kar di") when the operation failed;
- it reads out a raw ISO date, is too long for speech, or is in the wrong language;
- (rewording) it adds or drops a number, time, day or doctor, adds or removes a "nahi", drops the question the caller must answer or asks a new one, claims something was done, or turns the sentence onto the receptionist herself.

On rejection, on a missing or empty answer, or if Ollama is down, the Dialogue Manager's template for the turn is spoken - in English, Roman Urdu or Urdu. The call never waits on or breaks because of the model.

10.4 Measured

Check (29 September 2026)	Result
Judge: 21 live calls (7 scenarios × 3 languages)	wording used 3 times; 18 fell back to the template (9 no usable sentence, 6 rejected by the validator, 3 other); median 1.11 s, max 6.23 s

Rewording: two runs of 21 conversation turns	15 and 13 accepted, 3 and 5 rejected (an invented "Friday", "kal" turned into "today", a slot dropped, an added "Nahin"), 3 judge turns; median 0.66-0.79 s per reply
Stage timing (<code>measure_performance.py</code>)	judge median 0.78 s (3 calls), rewording median 0.49 s (5 calls)

The validator checks facts, not grammar: an accepted rewording can still be clumsy ("Bilkul. Aap kis din convenient rahega?"). A larger local model would help (section 17). Before 29 September every Ollama request also waited about 2.1 s because the address was `localhost`, which Windows first tries over IPv6; the default is now `127.0.0.1` (measured 2,236-2,316 ms against 191 ms for the same request).

11. Doctor and SuperAdmin Dashboards

One web application (Dashboard/ , served by the API at /ui) with two roles. It is plain HTML, CSS and JavaScript with no framework and no build step, works in light and dark themes and on a phone-width screen, and refreshes its counters every 30 seconds. The screenshots in this chapter were taken from the running system on 29 September 2026 with the demo clinic and invented demo patients; all figures on them are computed from those database records.

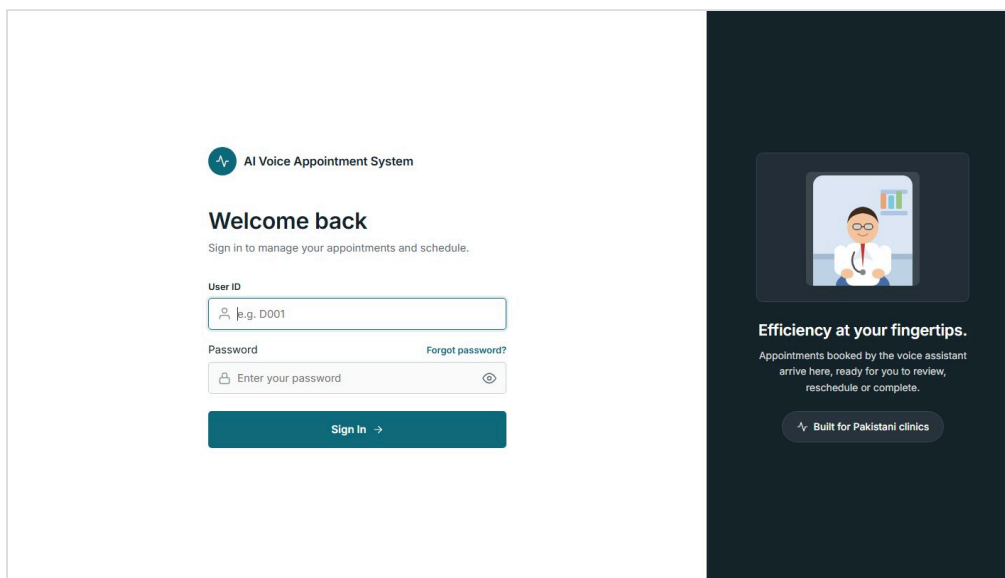


Figure 11.1 - Sign-in: a doctor ID or username, or the administrator ID, and a password.

11.1 Doctor Dashboard

- **Dashboard:** today's counters (appointments, completed, still to come, waiting now, cancelled), the doctor's own statistics for today / this week / this month / all time / a custom range, today's list with *view*, *complete* and *cancel*, and who is due now.
- **Appointments:** the doctor's own appointments with search and filters; book, move, cancel and complete through the Appointment Backend.
- **Patients:** the patients who have an appointment with this doctor, each with their history with this doctor; register a new patient.
- **Schedule:** the weekly working hours the assistant offers to callers (one or more sessions per day, days off) and **leave:** blocking dates cancels that day's appointments and queues a message to each patient.
- **Doctor Profile:** name, specialisation, qualification, experience, phone, email, fee, a short text and a photo. The clinic's details are shown but not editable; the username is shown but set by the administrator.
- **Notifications:** the messages queued for this doctor's patients, with read/unread.
- **Settings:** own password (the current one is required), alert preferences, theme.

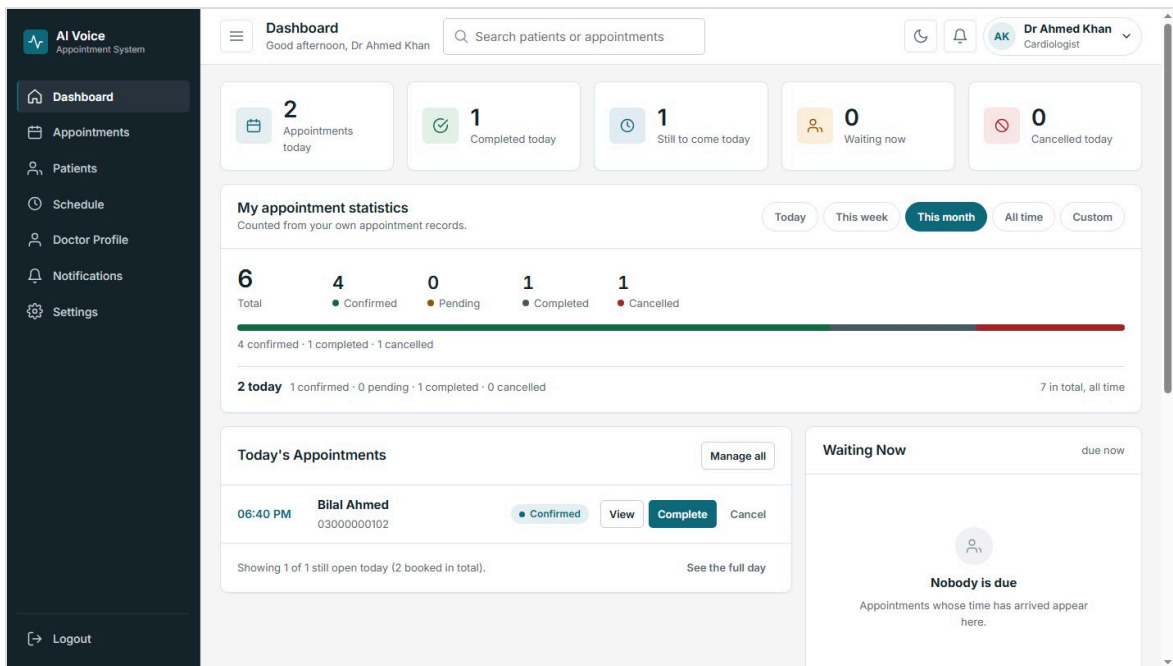


Figure 11.2 - Doctor Dashboard: today's counters, own statistics by period, today's appointments.

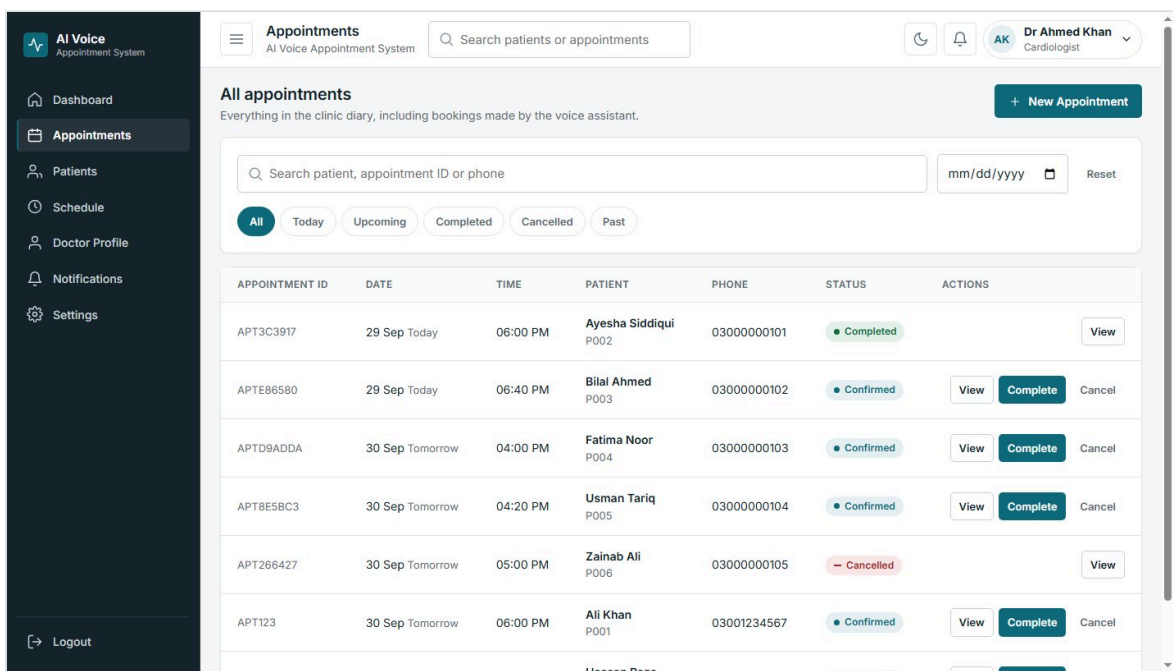


Figure 11.3 - The doctor's own appointments.

My Schedule
AI Voice Appointment System

Search patients or appointments

Dr Ahmed Khan
Cardiologist

My Schedule
The working hours the booking system offers to callers.

Weekly availability
Turning a day off stops new bookings on that weekday.

Save schedule

Day	Availability	Action
Monday	04:00 PM to 08:00 PM	+ Add session
Tuesday	04:00 PM to 08:00 PM	+ Add session
Wednesday	04:00 PM to 08:00 PM	+ Add session
Thursday	04:00 PM to 08:00 PM	+ Add session
Friday	04:00 PM to 08:00 PM	+ Add session
Saturday	04:00 PM to 08:00 PM	+ Add session
Sunday	Clinic closed	

Logout

Figure 11.4 - Weekly working hours; Sunday off. Leave is managed on the same page, below.

Settings
AI Voice Appointment System

Search patients or appointments

Dr Ahmed Khan
Cardiologist

Settings
Your account, alerts and how the dashboard looks.

Account

Upload photo

Shown beside your name in this dashboard.

Name
Dr Ahmed Khan

Email
name@clinic.pk

Contact phone
+923001234567

Username
D001

You sign in with this, or with your ID D001. Your clinic administrator sets the username.

Password

Current password

New password
At least 8 characters

Repeat it

Change password

Alerts
Shown while this dashboard is open.

New appointments
When the voice assistant books someone into today's diary.

Cancellations
When a patient cancels an appointment for today.

Changed times
When a patient moves an appointment to another time.

Messages to patients are a separate thing: the system queues them when you block a day, and the Notifications page shows that queue.

Logout

Figure 11.5 - Settings: own password; the username is set by the clinic administrator.

11.2 SuperAdmin Dashboard

- **Overview:** doctors on the register, bookable, deactivated and on leave today; today's and upcoming appointments; clinic-wide statistics by period with a per-doctor table - choosing a doctor narrows the figures.
- **Doctors:** add a doctor (with a username and password at the same time), edit, activate/deactivate (a deactivated doctor cannot be booked), remove (archive, keeping history) and restore; each doctor's detail page shows their schedule, leave, workload and sign-in details.
- **Credentials:** set or change a doctor's username and password; a password change signs that doctor out of any open session.

- **Leave:** block dates for any doctor after a preview of which patients will be affected; cancel the appointments on those dates and queue the patient messages.
- **Appointments:** every appointment in the clinic, including those booked by the voice assistant - upcoming, today, past, cancelled, all; search by patient, phone, doctor or ID; cancel, move or complete any of them.
- **Clinic:** the one clinic's name, address, city and phone, used by every screen and read to callers.
- **Settings:** the administrator's own password.

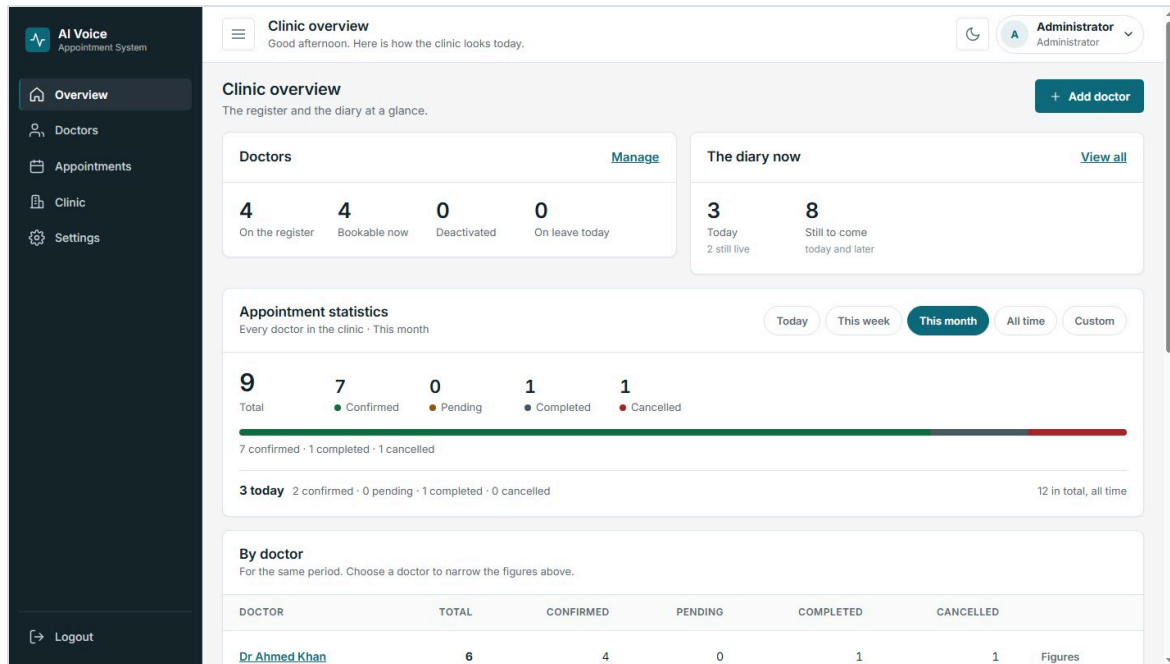


Figure 11.6 - SuperAdmin overview: register, diary, clinic-wide statistics and the per-doctor table.

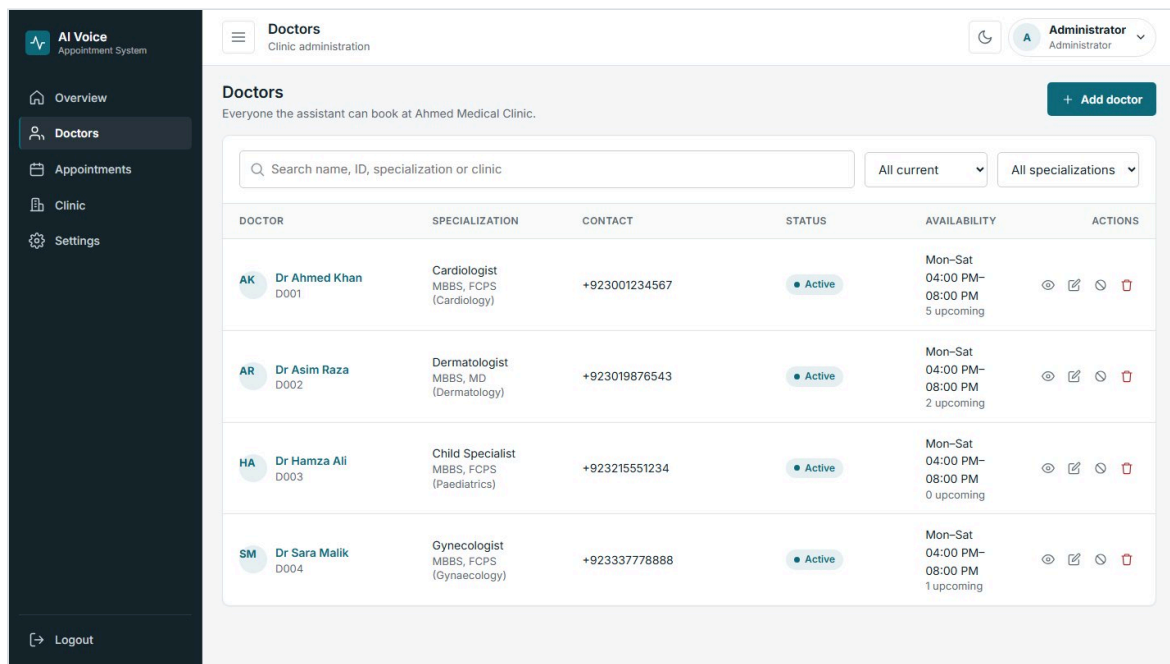


Figure 11.7 - Doctor management: view, edit, deactivate and remove.

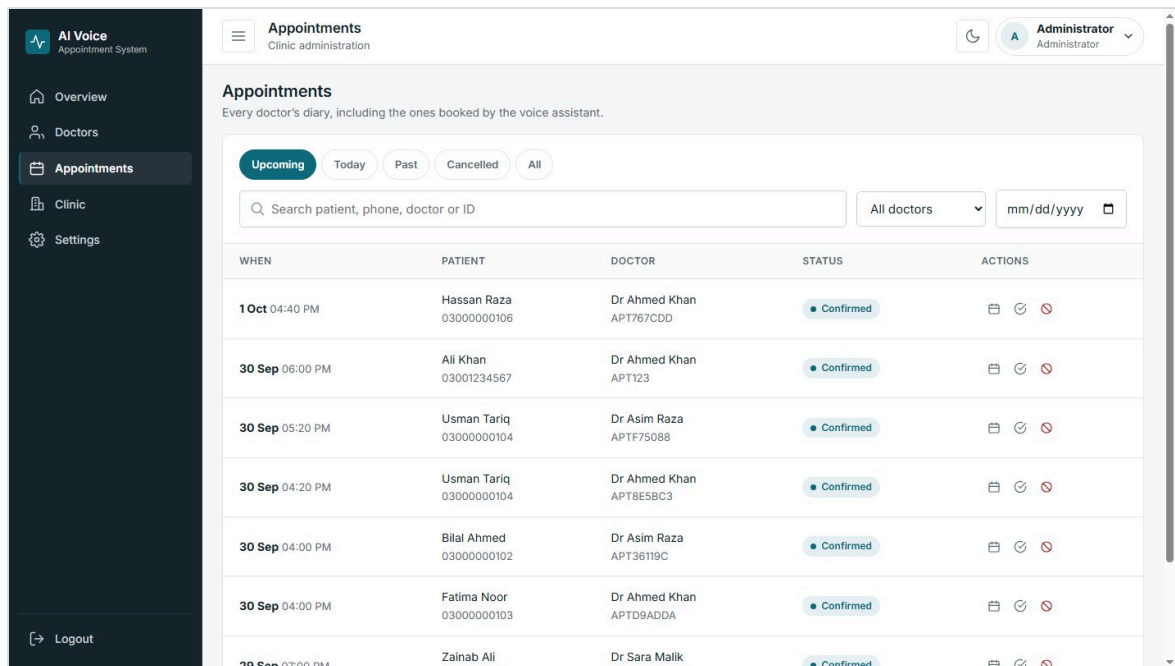


Figure 11.8 - Every doctor's appointments, with move, complete and cancel.

Clinic information
4 doctors work here

Clinic name
Ahmed Medical Clinic

Spoken to callers when the assistant confirms an appointment.

Address
Main Bazar Road, Sector G-10, Islamabad

City
Islamabad

Clinic phone
+925111234567

Save clinic details

Figure 11.9 - The single clinic's details - one record, no way to add another.

11.3 Access control

Enforced on the server, not by hiding buttons: `/dashboard/*` routes need a doctor's session and read the doctor ID from that session, never from the request; `/admin/*` routes need the administrator's session; each role is refused by the other's routes (`tests/test_admin_api.py` , `TestRoles`).

Action	SuperAdmin	Doctor
Add, edit, deactivate, remove, restore doctors	Yes	No
Change a doctor's username	Yes	No - not even their own

Change a doctor's password	Yes, any doctor	Own only, with the current password
Change own password	Yes	Yes
Clinic name, address, city, phone	Yes	No (403 clinic_readonly)
Working hours and leave	Any doctor	Own only
Appointments	All; cancel, move, complete	Own only; book, cancel, move, complete
Statistics	Clinic-wide and per doctor	Own only
Doctor profile (name, fee, specialisation...)	Any doctor	Own profile
Another doctor's data	Yes (administration)	No
Open a doctor's private dashboard	No - roles do not overlap	Own only

11.4 Statistics

Counted from the appointment records the caller may see - a doctor's own, or the whole clinic's for the administrator - never from sample values (`appointment_backend/statistics.py`). Figures: total, confirmed, pending, completed, cancelled (including `cancelled_by_doctor` , also reported on its own) and rescheduled, so the parts add up to the total exactly once. Periods: today, this week, this month, all time and a custom range; all-time totals use Firestore `count()` aggregations. The administrator also gets the same figures per doctor.

12. Security

Area	What the code does
Secrets	All credentials come from <code>.env</code> or the environment. <code>.env</code> , the Firebase service-account file and every JSON file not explicitly listed are ignored by Git, as are audio recordings and private voice-session reports. <code>.env.example</code> holds names only.
ElevenLabs key	Server-side only; sent to ElevenLabs in the <code>xi-api-key</code> header, never in a URL, a response, an event, a log line, the dashboard or Git (checked on 29 September 2026: in no changed file and not in the Git history).
Firebase credentials	Read by the API only; the browser never talks to Firebase and holds no credential or database address.
Authentication	Sign-in returns a random opaque token; every protected route needs <code>Authorization: Bearer</code> . Sessions last 12 hours; the browser keeps the token in <code>sessionStorage</code> , so it is gone when the tab closes.
Passwords	PBKDF2-HMAC-SHA256, 120,000 iterations, 16-byte random salt, in <code>dashboard_accounts</code> ; never returned by the API; at least 8 characters; changing one needs the current one and ends the account's other sessions. A deployment with no password configured refuses sign-in rather than being open.
Roles	Doctor and superadmin, enforced per route (section 11.3).
Call endpoint	<code>/voice/call</code> needs <code>VOICE_GATEWAY_TOKEN</code> ; with none set it accepts only local connections.
Logging	Speech logs record transcript length, not words (words only at <code>DEBUG</code> level). The <code>websockets</code> library's debug output, which contains request headers, is suppressed.
Checks	<code>tests/test_security.py</code> scans the code, the reports and this PDF for private keys and service-account addresses and checks the secret files are ignored; <code>scripts/build_report.py</code> refuses to publish a PDF that contains credential material.

Known limits. Sessions live in the API process, so a restart signs everyone out; there is no sign-in rate limiting; and the text dialogue routes (`/dialog/*`, `/voice/message`) are unauthenticated because they are meant to sit behind a telephony layer on a private network.

13. Testing

13.1 Automated test suite

`python scripts/run_tests.py`, run on 29 September 2026 with the trained model present. The offline tests use the in-memory database and simulated external services (ElevenLabs, Ollama, sound card) and never touch Firestore.

Module	Tests	Passed	Failed	Skipped	Status
Intent detection (mBERT, router, Roman Urdu, audit sets)	45	45	0	0	Pass
Dialogue (Dialogue Manager, entities, naturalness, scenarios, emergency net)	112	112	0	0	Pass
Appointment backend	61	61	0	0	Pass
Firebase (repository, services)	40	31	0	9	Pass live tests skipped
Ollama judge, validator, languages	54	51	0	3	Pass live tests skipped
End-to-end (pipeline, HTTP API)	25	25	0	0	Pass
Doctor Dashboard API	28	27	0	1	Pass
SuperAdmin API (doctors, credentials, clinic, leave, roles)	70	70	0	0	Pass
Statistics	24	24	0	0	Pass
Database performance budgets	31	31	0	0	Pass
STT (realtime, batch, keyterms)	13	13	0	0	Pass
TTS	7	7	0	0	Pass
Voice calls end to end, audio, known confusions	35	35	0	0	Pass
Microphone voice mode	39	39	0	0	Pass
Security	11	11	0	0	Pass
Total	595	582	0	13	No failures

Twelve of the 13 skipped tests need `--live` (the real Firestore project and Ollama); one dashboard test skips itself because the backend refuses the set-up it needs (two same-day bookings for one patient). Without the trained model (a fresh clone) the suite gives 589 tests, 534 passed, 55 skipped, 0 failed: the tests that need the real model skip themselves. The dialogue tests use a scripted stand-in classifier on purpose, so they test the dialogue logic on its own.

13.2 Browser tests and live checks

- **Browser:** two scripts drive the real dashboard in headless Chrome (`tests/browser/`) - signing in as each role, every page, adding and editing doctors, credentials, the clinic record, leave with its preview, appointment actions, statistics, the phone-width layout, and no JavaScript error: **Doctor Dashboard 59 of 59, SuperAdmin 51 of 51** (29 September 2026).

- **Ollama, live:** `tests/test_ollama_live.py` , 3 tests passed (29 September 2026).
- **Firestore, live:** the booking/cancel/leave tests against the live project were not re-run for this report because they write to it; read-only timings were measured instead (section 14).
- **Speech, live:** measured with the real ElevenLabs service and synthetic callers (section 5).

13.3 Important test cases - actual system output

Run through the real pipeline (mBERT, Dialogue Manager, backend) on the in-memory demo clinic, replies following the caller's language, on 29 September 2026. "Kal" is 30 September.

Case	Caller said	mBERT	System reply (action)
Critical sentence	Mje docter ke pas jana h	book_appointment 1.00	"Which doctor would you like to see?" (ask_for_doctor) - see note
Booking	Mujhe Dr Asim se appointment chahiye → kal → 5 baje → haan kar dein	book 1.00 ... confirm 0.99	"Ji. Dr Asim Raza ke paas kal shaam 5 baje ka slot khali hai. Book kar doon?" → "Done ji. Aap ki appointment ... confirm ho gayi hai." (created)
Slot taken	I want to book Dr Asim tomorrow at 5 PM	book_appointment 1.00	"5:00 PM is already taken. Dr Asim Raza has 4:20 PM, 4:40 PM or 5:20 PM free - which suits?"
Booking, Urdu script	مجھے ڈاکٹر سارہ سے کل چار ہاں → بجے اپائنٹمنٹ چاہیے	book 1.00, confirm 0.99	"جی ہو گئی۔ آپ کی اپائنٹمنٹ Dr Sara Malik کے ساتھ کل شام 4 بجے کنفرم ہو گئی ہے۔" (created)
Doctor fee	Dr Ahmed ki fee kitni hai?	doctor_fee 0.99	"Dr Ahmed Khan ki consultation fee 2000 rupay hai." (from the database)
Availability	Dr Hamza kal available hain?	check_availability 0.99	"kal Dr Hamza Ali ke paas shaam 4 baje, shaam 4:20 baje ya shaam 4:40 baje available hai..."
Appointment lookup	Meri appointment kab hai?	appointment_status 0.96	"Aap ki appointment Dr Ahmed Khan ke saath kal shaam 6 baje par hai."
Rescheduling	I want to reschedule my appointment → APT123 → Friday → 5 PM → yes	reschedule 0.99 ...	"Done, I have moved it to Friday, 2 October at 5:00 PM." (moved)
Cancellation	Meri appointment cancel kar dein → APT123 → haan	cancel 0.99 ... confirm 0.99	"Ji, aap ki Jumma 2 tareekh wali appointment Dr Ahmed Khan ke saath cancel kar di hai."
Correction	Dr Ahmed se appointment chahiye → nahi Dr Asim → parson → 4 baje	...	"Acha. Dr Asim Raza ke paas parso shaam 4 baje ka slot khali hai. Book kar doon?"
Emergency	Mere abbu ko seene mein dard hai, saans nahi aa rahi	emergency 0.96	"Yeh emergency lag rahi hai. Foran emergency services ko call karein..." (escalate)
Goodbye	Shukriya, Allah Hafiz	thank_you 0.98	"Khuda hafiz, apna khayal rakhiye ga." (conversation ended)

Note on the critical sentence. The intent is right, and a live voice test with ElevenLabs gave the same intent. With the reply language on automatic, the language detector did not recognise “Mje docter ke pas jana h” as Roman Urdu and answered in English. `RESPONSE_LANGUAGE=roman_urdu` fixes the reply language for a deployment; the detection itself is a limitation (section 17).

Also covered by the automated tests: doctor leave cancelling that day's appointments and queuing one notification per patient; the SuperAdmin editing a doctor, changing credentials and the clinic; a doctor refused when changing the clinic, another doctor's data or a username; and whole voice conversations (booking, cancellation, rescheduling, fee, availability, goodbye, a caller interrupting, a dropped connection, a failing TTS service) through the real pipeline.

13.4 Problems found and fixes

Problem	Root cause	Fix	Status
“Mje docter ke pas jana h” → <code>change_doctor</code> 0.41 → “please repeat”	Informal spelling; an unsure model sent to “unknown”	Booking corroboration rule; Roman Urdu spelling variants added to training	Fixed: now <code>book_appointment</code> 1.00
“Bohot khoon beh raha hai” → <code>thank_you</code> 0.97	Model error on an unseen emergency	Emergency safety net on danger phrases	Fixed, tested
30 audit and 1 hold-out sentence were also in the training data	Merging several datasets	Removed; group-aware split; retrained	Fixed; honest scores
“dr ahmad” during a booking → asked for an appointment ID, repeatedly	<code>change_doctor</code> treated as a reschedule	Draft-edit labels edit the booking in progress	Fixed, tested
“nahi” to “appointment ID?” looped	No path for a caller without an ID	Use the caller's own appointment(s)	Fixed
A fee question during a booking was ignored	The doctor's name was taken as a booking slot	Answer the side question, then resume	Fixed
LLM said “kal 5 baje” for an appointment 8 days away	Validator checked times and IDs only	Spoken-date and reply-language checks	Fixed
Doctor sign-in took 7.3 s	Sequential reads of whole collections	Narrow, batched, parallel, counted and paged reads; cache	1.2 s
Calls lost a turn after a long agent reply	ElevenLabs closes a session after ~15 s without audio	Keep-alive silence; re-send the sentence in progress	Fixed, tested
“ہاں، کر دیں” (yes) not taken as yes	Urdu comma not stripped; mBERT read it as <code>change_date</code>	Strip Urdu punctuation; a yes to a pending booking stays a yes	Fixed, tested
Agent's reply cut off mid-sentence	A late partial transcript treated as the caller interrupting	Interrupt only when the caller is also making sound	Fixed; test fails without it
The API key appeared in a debug log	<code>websockets</code> logs request headers at DEBUG	Logger held at INFO	Fixed; test enforces

First caller after a restart waited 10.3 s	mBERT loaded on first use	Load at start-up in the background	Fixed
Every Ollama request took 2.1 s longer	<code>localhost</code> tried over IPv6 first on Windows	Default address <code>127.0.0.1</code>	Judge median 4.12 s → 1.11 s
Loading mBERT could break other imports	The intent module's own <code>config</code> briefly replaced the project's, process-wide	Load the intent module under private names	Fixed; found by the test suite
TTS changed "4 baje" to "5 baje"	Flash v2.5 model	Chose <code>eleven_v3_conversational</code> by measurement	Fixed

14. Performance

Measured on the development machine (GTX 1660 SUPER, network in Pakistan); sources:

`reports/performance.json` , `reports/ollama_live.json` , `reports/voice/` , `docs/PERFORMANCE.md` ,
`docs/VOICE.md` .

Stage	Measured	Date
STT: caller stops → final transcript	1.5-1.8 s (0.8 s of it the end-of-sentence pause); session connect $\approx$ 0.8 s	29 Sep
mBERT inference	median 7.5 ms, p95 8.7 ms (60 sentences, GPU); first call including model load 17.9 s	29 Sep
Dialogue Manager logic	median 0.31 ms	29 Sep
Whole text turn, no LLM, in-memory database	median 7.9 ms	29 Sep
Backend booking, in-memory	median 0.09 ms	29 Sep
Firestore, live, read-only	get a doctor: median 2.40 s (5 calls); availability: median 4.07 s (5 calls) - one round trip is 0.55 s or $\approx$ 2.35 s from this network	29 Sep
Ollama judge	median 1.11 s over 21 calls (max 6.23 s)	29 Sep
Ollama rewording (voice mode)	median 0.49 s (5 calls); per-conversation medians 0.86-0.97 s in the live microphone runs	29 Sep
TTS: request → first audio	median 824 ms (<code>e11even_v3_conversational</code>), 818 ms (Flash v2.5); 0.65-0.70 s inside calls	29 Sep
Caller stops → agent speaks, calls (judge off)	median 2.52 s, 20 turns (2.30-2.98 s)	29 Sep
Caller stops → agent speaks, microphone mode (Ollama every reply)	median 3.05 s, 18 turns; booking confirmations 4.2-7.9 s (the judge)	29 Sep
Voice turn against live Firestore	Not measured - the voice measurements used the in-memory clinic	-
Doctor sign-in to a usable dashboard	1.2 s (was 7.3 s)	28-29 Sep
Dashboard 30-second refresh	0.57 s, 1 request (was 3.4 s, 4 requests)	28-29 Sep
Administrator overview	1.7 s (was 5.7 s)	28-29 Sep
Telephony	Not measured - not implemented	-

Every stage of a voice turn is shown live in `voice_app.py --debug` and saved with `--report` .

15. User Flows

15.1 Booking

```
Patient: "Mujhe Dr Asim se appointment chahiye"  
  → STT → mBERT book_appointment (1.00) → Dialogue Manager: doctor = Dr Asim Raza; date missing  
  → "Bilkul. Kis din aana convenient rahega?"  
Patient: "kal"          → date = tomorrow; time missing → "Kis waqt aana chahenge?"  
Patient: "5 baje"       → time = 17:00 → Backend: availability check (no write)  
  → "Ji. Dr Asim Raza ke paas kal shaam 5 baje ka slot khali hai. Book kar doon?"  
Patient: "haan kar dein" → Backend: gates + transactional write → Firestore: appointment confirmed  
  → reply worded by Ollama, checked by the validator (or the template)  
  → "Done ji. Aap ki appointment Dr Asim Raza ke saath kal shaam 5 baje confirm ho gayi hai.  
    Appointment ID ... hai." → TTS → patient; visible in both dashboards
```

15.2 Cancellation

```
Patient: "Meri appointment cancel kar dein" → cancel_appointment (0.99)  
  → the caller's appointment is found → "... Dr Ahmed Khan ke saath Jumma 2 tareekh shaam 5 baje hai.  
    Cancel kar doon?"  
Patient: "haan" → Backend: exists, belongs to the caller, still live → status cancelled  
  → "Ji, aap ki Jumma 2 tareekh wali appointment Dr Ahmed Khan ke saath cancel kar di hai."
```

15.3 Rescheduling

```
Patient: "I want to reschedule my appointment" → reschedule_appointment (0.99)  
  → appointment ID (or the caller's appointment) → new date → new time  
  → Backend: slot free? → "Dr Ahmed Khan is free Friday, 2 October at 5:00 PM. Shall I move your  
    appointment?"  
Patient: "yes" → status rescheduled, new date and time → "Done, I have moved it to Friday, 2  
    October at 5:00 PM."
```

15.4 Doctor unavailable

```
SuperAdmin → Doctors → Dr Hamza Ali → Leave → 1 October, "Medical conference" → the preview lists  
the 2 affected patients  
  → confirm → 2 appointments cancelled_by_doctor, 2 notifications queued (not sent)
```


16. Project Structure, Installation and Libraries

16.1 Structure

```
AI-voice-appointment-system/
├── api/                FastAPI: main.py (app, dialogue routes, /ui), auth.py, dashboard.py,
│                       admin.py, data.py (dashboard reads), voice_ws.py (/voice/call)
├── appointment_backend/ booking, cancellation, rescheduling, availability, validation,
│                       statistics, public appointment routes
├── dialog_manager/     Dialogue Manager, intent router, entities, slots, sessions, fallbacks
├── ollama_judge/        Ollama client, prompts, languages and templates, response validator
├── speech/             ElevenLabs STT and TTS, audio, call orchestration, microphone, speaker
├── firebase/           Firestore repository, one service per collection, cache, seed data
├── intent_detection/    mBERT: src/ (preprocess, train, evaluate, inference), data/,
│                       outputs/ (metrics, figures), models/ (trained weights, not in Git)
├── Dashboard/          the Doctor and SuperAdmin web app (HTML, CSS, JavaScript)
├── Data/               raw intent datasets
├── docs/               this report (PDF and its HTML source), VOICE.md, PERFORMANCE.md
├── reports/            evaluation, test, performance and voice measurement outputs
├── scripts/            run_tests, build_report, voice_check, voice_benchmark,
│                       measure_performance, evaluate_intents, firestore_indexes, ...
├── tests/              unittest suite; browser/ (headless-Chrome checks)
├── config.py           every setting, from environment variables
├── voice_app.py         microphone voice mode
├── voice_pipeline.py    text console for the full pipeline
├── firestore.indexes.json, firebase.json, requirements.txt, .env.example, README.md
```

16.2 Installation requirements

Requirement	Needed for
Python 3.12	everything (tested 3.12.10); <code>pip install -r requirements.txt</code>
The trained mBERT model	intent detection (712 MB, not in Git; train it with <code>intent_detection/src/train.py</code> or copy it in)
NVIDIA GPU (optional)	training (strongly recommended) and fast inference; the system also runs on a CPU (not measured)
Firebase project with Firestore (optional)	persistent data; without it an in-memory database with demo data is used
Ollama with <code>llama3.2</code> (optional)	reply wording; without it templates are used (<code>OLLAMA_ENABLED=false</code>)
ElevenLabs account, API key and voice ID	speech only; typed text works without it
Microphone and speaker or headphones	microphone mode (PortAudio is bundled with <code>sounddevice</code> on Windows)
Node.js 22+ and Google Chrome	browser tests and rebuilding this PDF only
Asterisk	not needed - telephony is not implemented

Start the API with `uvicorn api.main:app --port 8000` and open `http://127.0.0.1:8000/ui/` ; talk by voice with `python voice_app.py` . The README gives every step.

16.3 Libraries (`requirements.txt`)

Package	Minimum	Used for
torch	2.4	running and training mBERT
transformers	4.44	mBERT model and tokenizer
pandas, numpy, scikit-learn, matplotlib	2.2 / 1.26 / 1.5 / 3.8	dataset preparation, metrics, training plots
firebase-admin, google-cloud-firestore	6.5 / 2.16	Firestore
fastapi, uvicorn[standard], pydantic	0.115 / 0.30 / 2.8	HTTP API
httpx	0.27	Ollama and ElevenLabs TTS client; API tests
websockets	12	ElevenLabs realtime speech-to-text
sounddevice	0.4	microphone and speaker
python-dotenv	1.0	loading <code>.env</code>
pypdf	4.0	checking this PDF for secrets

There is no `package.json` : the dashboard has no JavaScript dependencies and the browser tests use only Node's built-in modules. The official ElevenLabs SDK is not used (its install fails on the development machine's Windows path limit); the documented REST and WebSocket APIs are called directly.

17. Limitations and Future Work

17.1 Limitations

- **No telephone line.** Asterisk/SIP is not implemented; the system is reached by microphone, WebSocket or text.
- **Speech not tested with real people.** All speech measurements used synthetic callers (ElevenLabs voices), which mis-heard Urdu time words ("kal" → "Cole", "kal ke liye" → "Calcutta"). Noise, accents and real microphones are untested.
- **Voice.** On the free ElevenLabs plan only built-in voices work through the API; the agent's voice is an English voice speaking Urdu, and its pronunciation has not been judged by a listener. Speech needs an internet connection and costs credits per use.
- **Intent model.** Most of the corpus (2,374 of 3,461 sentences) is generated; several intents have fewer than ten test sentences; goodbye, clinic timing, help and repeat are weak; there is no out-of-scope class ("Aaj mausam kaisa hai?" is answered with clinic information).
- **Language detection** misses very informal Roman Urdu ("Mje docter ke pas jana h" was answered in English with the reply language on automatic).
- **Language model.** llama3.2 (3B) wording was used for 3 of 21 booking results and for 13-15 of 18 other replies in two runs; the validator checks facts, not grammar; it adds about 0.5-1 s per reply and occasionally several seconds.
- **Notifications** are queued but not delivered (no SMS or call provider).
- **Firestore latency** from the development network is 0.55-2.35 s per round trip, the composite indexes are not deployed, and a voice turn against live Firestore has not been measured.
- **Rescheduling** re-checks the slot and then updates the record, but not inside a transaction, unlike a new booking.
- **Authentication** is a prototype: in-process sessions, no rate limiting, unauthenticated dialogue routes.
- **Microphone mode through loudspeakers** cannot be interrupted (the microphone is muted while the agent speaks).
- The text console (`voice_pipeline.py`) re-writes the demo doctors, schedules and patients into the configured database when it starts; against Firestore that overwrites dashboard edits to those records.

17.2 Future work

- An Asterisk bridge (ARI External Media or AudioSocket) to the existing `/voice/call` endpoint, and tests over a real phone line.
- Testing with real patients' speech and recordings; an Urdu-native voice (a paid ElevenLabs plan).
- More natural training sentences per intent, a fresh held-out set, and an out-of-scope class.
- A stronger local model (for example `qwen2.5:7b` , which fits the 6 GB GPU) for better Roman Urdu wording.
- An SMS or voice-call provider behind the existing `NotificationSender` interface.
- Firebase Authentication and rate limiting; deploying the declared Firestore indexes; a transactional reschedule; fewer sequential Firestore reads in the booking path.
- A mobile-friendly dashboard application, richer analytics, and a production deployment (hosting, HTTPS, monitoring).

18. Conclusion

The project delivers a working AI appointment assistant for a single clinic. A patient can book, check, reschedule and cancel an appointment by speaking English, Urdu, Roman Urdu or a mix, through the computer's microphone and speaker today and through a telephone bridge once one is connected to the existing call endpoint. The same conversation works over text.

The main AI contributions are a fine-tuned mBERT classifier for 31 intents in mixed-script Pakistani speech (accuracy 0.898, macro-F1 0.836 on clean held-out data), a context-aware Dialogue Manager that keeps a conversation on track even when single words are misclassified, and a design in which a local language model improves the wording of replies while a validator guarantees that every fact the patient hears comes from the database. Appointment automation is deterministic and safe: validation gates, transactional booking, alternatives when a slot is taken, and a doctor-leave cascade that cancels and records notifications for every affected patient.

Doctors manage their own day, and the administrator manages the doctors, their credentials and leave, every appointment and the clinic itself, with role rules enforced on the server and statistics computed from real records. The system is tested by 595 automated tests and 110 browser checks with no failures.

It is ready for demonstration and supervised trials. Before real patients can rely on it by telephone, it needs the Asterisk/SIP bridge, testing with real callers, delivery of patient notifications and production-grade authentication.

References

1. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019. arXiv:1810.04805.
2. Google Research, "Multilingual BERT," github.com/google-research/bert/blob/master/multilingual.md.
3. T. Pires, E. Schlinger and D. Garrette, "How multilingual is Multilingual BERT?," in *Proc. ACL*, 2019. arXiv:1906.01502.
4. Y. Wu et al., "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," arXiv:1609.08144, 2016 (WordPiece).
5. T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proc. EMNLP: System Demonstrations*, 2020. Hugging Face Transformers documentation: huggingface.co/docs/transformers.
6. A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*, 2019.
7. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
8. Google, "Cloud Firestore documentation," firebase.google.com/docs/firestore; "Firebase Admin SDK," firebase.google.com/docs/admin/setup.
9. ElevenLabs, "Speech to Text" and "Text to Speech" API documentation, elevenlabs.io/docs.
10. Ollama, ollama.com and github.com/ollama/ollama.
11. Meta, "Llama 3.2 model card," github.com/meta-llama/llama-models.
12. S. Ramírez, "FastAPI," fastapi.tiangolo.com.
13. M. Geier, "python-sounddevice," python-sounddevice.readthedocs.io; PortAudio, portaudio.com.
14. Sangoma, "Asterisk," asterisk.org (planned telephony).
15. Google, "Chrome DevTools Protocol," chromedevtools.github.io/devtools-protocol (browser tests).

Appendix A – HTTP API

Route	Purpose
Conversation (for a telephony layer on a private network)	
POST /dialog/message , POST /voice/message	one caller turn as text → reply and full turn details
POST /dialog/reset , POST /dialog/end , GET /dialog/transcript/{id}	session control and transcript
POST /intent/predict	mBERT classification only
WS /voice/call	a whole voice call (audio in, audio and events out); token required
Appointments (Appointment Backend)	
POST /appointments/book , /cancel , /reschedule , /check	operations; /check writes nothing
GET /appointments/{id} ; patient and doctor listings; availability; doctor unavailability	reads
Sign-in	
POST /auth/login , /auth/logout , /auth/password ; GET /auth/session	sessions and own password
Doctor Dashboard (role doctor, own data only)	
GET /dashboard/summary , /statistics	counters, today, statistics
/dashboard/appointments (+ /cancel , /reschedule , /complete)	own appointments
/dashboard/patients , /availability , /profile , /schedule , /leave , /notifications	patients, free slots, profile, hours, leave, messages
SuperAdmin (role superadmin)	
GET /admin/summary , /statistics	clinic overview and statistics
/admin/doctors (+ /status , /restore , DELETE)	register: add, edit, activate/deactivate, archive, restore
/admin/doctors/{id}/credentials	a doctor's username and password
/admin/doctors/{id}/schedule , /leave , /leave/preview , /appointments , /availability	a doctor's hours, leave, diary
/admin/appointments (+ /cancel , /reschedule , /complete)	every appointment
GET/PATCH /admin/clinic , POST /admin/clinic/consolidate	the one clinic
Other	
GET /health , GET /doctors , /ui , /docs	status, public doctor list, dashboard, interactive API documentation

Appendix B – Configuration

Set in `.env` (template: `.env.example`). Names and defaults only - no values of secrets appear in this report.

Group	Variables (default)
Firebase	<code>FIREBASE_PROJECT_ID</code> , <code>FIREBASE_CLIENT_EMAIL</code> , <code>FIREBASE_PRIVATE_KEY</code> or <code>FIREBASE_CREDENTIALS_FILE</code> ; <code>USE_LOCAL_DB</code>
Clinic	<code>CLINIC_ID</code> (C001), <code>TIMEZONE</code> (Asia/Karachi), <code>APPOINTMENT_ID_PREFIX</code> (APT), <code>NOTIFICATION_PROVIDER</code>
Intent and dialogue	<code>CONFIDENCE_THRESHOLD</code> (0.60), <code>HIGH_CONFIDENCE</code> (0.80), <code>INTENT_SWITCH_THRESHOLD</code> (0.75), <code>EMERGENCY_SAFETY_NET</code> (true), <code>SESSION_TIMEOUT_MINUTES</code> (15), <code>RESPONSE_LANGUAGE</code> (auto), <code>INTENT_MODULE_DIR</code>
Ollama	<code>OLLAMA_BASE_URL</code> (http://127.0.0.1:11434), <code>OLLAMA_MODEL</code> (llama3.2), <code>OLLAMA_TIMEOUT</code> (30), <code>OLLAMA_TEMPERATURE</code> (0.2), <code>OLLAMA_MAX_TOKENS</code> (200), <code>OLLAMA_ENABLED</code> (true), <code>OLLAMA_TRANSLATE</code> (false)
Dashboards	<code>DASHBOARD_PASSWORD</code> , <code>DASHBOARD_PASSWORD_<DOCTORID></code> , <code>DASHBOARD_SESSION_HOURS</code> (12), <code>DASHBOARD_ORIGINS</code> , <code>SUPERADMIN_ID</code> (ADMIN), <code>SUPERADMIN_PASSWORD</code> - starting passwords, replaced by hashed ones once changed
Speech	<code>ELEVENLABS_API_KEY</code> , <code>ELEVENLABS_VOICE_ID</code> , <code>ELEVENLABS_STT_MODEL</code> (scribe_v2_realtime), <code>ELEVENLABS_STT_LANGUAGE</code> (ur), <code>ELEVENLABS_STT_AUDIO_FORMAT</code> (ulaw_8000), <code>ELEVENLABS_STT_SILENCE_SECS</code> (0.8), <code>ELEVENLABS_STT_KEYTERMS</code> (false), <code>ELEVENLABS_TTS_MODEL</code> (eleven_v3_conversational), <code>ELEVENLABS_TTS_OUTPUT_FORMAT</code> (ulaw_8000), <code>ELEVENLABS_TIMEOUT</code> (15), <code>ELEVENLABS_MAX_RETRIES</code> (1), <code>ELEVENLABS_ENABLE_LOGGING</code> (true), <code>VOICE_GATEWAY_TOKEN</code>
Microphone mode	<code>MICROPHONE_DEVICE</code> , <code>SPEAKER_DEVICE</code> (system default), <code>VOICE_INPUT_MODE</code> (vad), <code>VOICE_DEBUG</code> (false), <code>VOICE_MIC_FORMAT</code> (pcm_16000), <code>VOICE_TTS_OUTPUT_FORMAT</code> (pcm_22050), <code>VOICE_BARGE_IN</code> (false), <code>VOICE_OLLAMA_EVERY_TURN</code> (true)
Development	<code>LOG_LEVEL</code> (INFO), <code>PERF_DEBUG</code>